ASSIGNMENT 7 SIMULATION OF DOMAIN NAME SERVER USING UDP	**Name:** Nivetha Dhanakoti **Reg no.:** 3122 22 5001 087

Aim:
To be proficient in developing an application to simulate domain name server using socket programming in C

Write the code using socket programming in C
Server should perform the following:
1. Maintain a DNS in the form of table. The table contains IP address and the corresponding Server name and displays the table.
2. When a request is for an IP address (Given a server name), from a client is received, verify the table and send the corresponding IP address to the client.
3. Make server to accept multiple client request simultaneously using UDP.
4. Also modify the server.

Client should do the following:
1. Request for an IP address is given to the server by the domain name.
2. Receive the corresponding IP address and display it.

Sample Input and Output
Server
Server Name IP Address
www.yahoo.com 10.2.45.67
www.annauniv.edu 197.34.53.122
www.google.com 142.89.78.66
Do you want to modify (yes or no): yes
Domain name: www.yahoo.com
IP address: 300.8.35.79
Invalid IP address
Enter valid IP address: 197.34.53.122
IP address already exists
Enter valid IP address: 196.34.53.122
Updated table is
Server Name IP Address
www.yahoo.com 10.2.45.67
196.34.53.122
www.annauniv.edu 197.34.53.122
www.google.com 142.89.78.66

Client1
Enter the server name: www.annauniv.edu
The IP address is :197.34.53.122

Client2
Enter the server name: www.yahoo.edu
The IP address is :10.2.45.67
Enter the server name: www.yahoo.edu
The IP address is :10.2.45.67
196.34.53.122

Server Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define MAX 1024
#define PORT 7676
#define MAX_IP_HISTORY 10 // Maximum history of IP addresses to store

typedef struct {
   char domain_name[50];
   char ip_history[MAX_IP_HISTORY][50]; // Store history of IP addresses
   int ip_count; // Number of IP addresses in history
} DNS;

DNS dns_table[] = {
   {"www.yahoo.com", {"10.2.45.67"}, 1},
   {"www.annauniv.edu", {"197.34.53.122"}, 1},
   {"www.google.com", {"142.89.78.66"}, 1},
};

int table_size = 3;

void display_table() {
   printf("\nServer Name\t\tIP Address\n");
   for (int i = 0; i < table_size; i++) {
      printf("%s\t", dns_table[i].domain_name);
      for (int j = 0; j < dns_table[i].ip_count; j++) {
         printf("%s ", dns_table[i].ip_history[j]);
      }
      printf("\n");
   }
}

int find_ip_by_domain(const char* domain, char* ip) {
   for (int i = 0; i < table_size; i++) {
      if (strcmp(dns_table[i].domain_name, domain) == 0) {
         strcpy(ip, "");
         for (int j = 0; j < dns_table[i].ip_count; j++) {
            strcat(ip, dns_table[i].ip_history[j]);
            if (j != dns_table[i].ip_count - 1) {
               strcat(ip, ", ");
            }
         }
         return 1;
      }
   }
   return 0;
}

int valid_ip(char* ip) {
   struct sockaddr_in sa;
```

```
    return inet_pton(AF_INET, ip, &(sa.sin_addr)) != 0;
}

int check_ip_exists(char* ip) {
    // Loop through the entire DNS table
    for (int i = 0; i < table_size; i++) {
        for (int j = 0; j < dns_table[i].ip_count; j++) {
            // If the IP address is already in the history, return 1
            if (strcmp(dns_table[i].ip_history[j], ip) == 0) {
                return 1;
            }
        }
    }
    return 0; // IP does not exist
}

void update_table() {
    char domain[50], new_ip[50];
    printf("\nDo you want to modify the table (yes or no): ");
    char response[5];
    scanf("%s", response);
    if (strcmp(response, "yes") == 0) {
        printf("Domain name: ");
        scanf("%s", domain);

        int found = 0;
        int domain_index = -1;

        // Find the domain in the DNS table
        for (int i = 0; i < table_size; i++) {
            if (strcmp(dns_table[i].domain_name, domain) == 0) {
                found = 1;
                domain_index = i;
                break;
            }
        }

        if (!found) {
            printf("Domain not found in table.\n");
            return;
        }

        while (1) {
            printf("IP address: ");
            scanf("%s", new_ip);

            if (!valid_ip(new_ip)) {
                printf("Invalid IP address, enter a valid IP.\n");
                continue;
            }

            // Check if the IP address already exists in the table
            if (check_ip_exists(new_ip)) {
                printf("IP address already exists in the table.\n");
                continue; // Ask for another IP
```

```
            }

            // Add the new IP address to the history
            if (dns_table[domain_index].ip_count < MAX_IP_HISTORY) {
                strcpy(dns_table[domain_index].ip_history[dns_table[domain_index].ip_count],
new_ip);
                dns_table[domain_index].ip_count++;
                printf("Updated table:\n");
                display_table();
                break;
            } else {
                printf("IP history limit reached, cannot add more IP addresses.\n");
                break;
            }
        }
    }
}


int main() {
    int sockfd;
    struct sockaddr_in servaddr, cliaddr;
    char buffer[MAX];
    char ip_addresses[MAX];

    // Create socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    // Server address
    memset(&servaddr, 0, sizeof(servaddr));
    memset(&cliaddr, 0, sizeof(cliaddr));

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    // Bind the socket
    if (bind(sockfd, (const struct sockaddr*)&servaddr, sizeof(servaddr)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    // Display DNS table
    display_table();
    update_table();

    while (1) {
        socklen_t len = sizeof(cliaddr);
        int n = recvfrom(sockfd, buffer, MAX, 0, (struct sockaddr*)&cliaddr, &len);
        buffer[n] = '\0';
        printf("Client requested domain: %s\n", buffer);
```

```
        if (find_ip_by_domain(buffer, ip_addresses)) {
            sendto(sockfd, ip_addresses, strlen(ip_addresses), 0, (const struct sockaddr*)&cliaddr,
len);
        } else {
            char *not_found = "Domain not found";
            sendto(sockfd, not_found, strlen(not_found), 0, (const struct sockaddr*)&cliaddr, len);
        }
    }

    close(sockfd);
    return 0;
}
```

Client Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define MAX 1024
#define PORT 7676

int main() {
    int sockfd;
    struct sockaddr_in servaddr;
    char buffer[MAX];
    char domain_name[50];

    // Create socket
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    memset(&servaddr, 0, sizeof(servaddr));

    // Server address
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = INADDR_ANY;

    socklen_t len = sizeof(servaddr);

    // Get domain name from user
    printf("Enter the server name: ");
    scanf("%s", domain_name);

    // Send domain name to server
    sendto(sockfd, domain_name, strlen(domain_name), 0, (const struct sockaddr*)&servaddr,
sizeof(servaddr));

    // Receive IP address from server
    int n = recvfrom(sockfd, buffer, MAX, 0, (struct sockaddr*)&servaddr, &len);
```

```
    buffer[n] = '\0';

    printf("The IP addresses are: %s\n", buffer);

    close(sockfd);
    return 0;
}
```

Server Output:

```
nivethadhanakoti@Nivethas-MacBook-Pro A7 % gcc server.c -o server
nivethadhanakoti@Nivethas-MacBook-Pro A7 % ./server

Server Name             IP Address
www.yahoo.com   10.2.45.67
www.annauniv.edu        197.34.53.122
www.google.com  142.89.78.66

Do you want to modify the table (yes or no): yes
Domain name: www.yahoo.com
IP address: 300.8.35.79
Invalid IP address, enter a valid IP.
IP address: 197.34.53.122
IP address already exists in the table.
IP address: 196.34.53.122
Updated table:

Server Name             IP Address
www.yahoo.com   10.2.45.67 196.34.53.122
www.annauniv.edu        197.34.53.122
www.google.com  142.89.78.66
Client requested domain: www.annauniv.edu
Client requested domain: www.yahoo.com
```

Client Output:

```
nivethadhanakoti@Nivethas-MacBook-Pro A7 % gcc client.c -o client
nivethadhanakoti@Nivethas-MacBook-Pro A7 % ./client
Enter the server name:  www.annauniv.edu
The IP addresses are: 197.34.53.122
nivethadhanakoti@Nivethas-MacBook-Pro A7 % ./client
Enter the server name: www.yahoo.com
The IP addresses are: 10.2.45.67, 196.34.53.122
nivethadhanakoti@Nivethas-MacBook-Pro A7 %
```

Learning Outcomes:

I have learnt to develop application to simulate domain name server using socket programming in C